SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Total Marks:50**

Annexure A

CLASS TEST

Code of Conduct:

I _A. Midhuna_ (192521302) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Q. No	Understanding of Concepts (25)	Explanation (25)	Application (20)	Organization (15)	Accuracy(10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 30	/ 20	/ 15	/ 10	/ 100

Annexure A

CLASS TEST

Q1. Construct three address code for the following

if((a < b) and ((c < d) or (a > d))) then

z=x + y *z

else

z = z + 1

Q2.Translate (a < b) and (c < d) or (e < f) into three address statements using back patching.

Q3. Consider the arithmetic expression:

$A = (-A + B) * ((-A + B) * C)$

(i).Analyze the given expression and explain how operator precedence and associativity influence its evaluation order.

(ii). Construct the Three Address Code (TAC) for the expression by breaking it into intermediate steps using temporary variables.

(iii). Represent the generated TAC in the following forms:

- (a).Quadruples (op, arg1, arg2, result)
- (b).Triples (index, op, arg1, arg2)
- (c).Indirect Triples (pointer-based representation)

(iv). Compare the representations and analyze:

- (a).Memory usage
- (b).Ease of code optimization

(c).Advantages of indirect triples over triples

Q4. Construct three address code using the following production for Boolean expression (a > b) or (c < d) or (e < f)

B → B1 || B2

B → B1 && B2

B → ! B1

B → E1 relop E2

B → true

B → false

Answer:

CO4 AT1 class Test

Name : A. Midhuna

Register No : 192521302

Date : 18/8/26

39/40

1 construct the Address code for the following if (a<b) and (c<d) or (a>d) then z = x+y*z else z=z+1

Step1 : The structure in (a<b) and (c<d) or (a>d) create labels:

use labels for the condition which then part else part and end of the statement.

L1, L2, L3, L4, L5, L6

Generate TAC for the condition

```
L1 : if a<b goto L2
     goto L5

L2 : if c<d goto L4
     goto L3

L3 : if a>d goto L4
     goto L5
```

if a<b false the complete AND expression is false to control goes to the else part. if a<b in expression evaluated if either c<d or a>d true control goes to then part.

L4 : $t_1 = y * z$

$t_2 = x + t_1$

$z = t_2$

goto L6

L5 : $t_3 = z + 1$

$z = t_3$

Translate (a<b) and (c<d) or (e<f) into three Statements using back patching.

(a<b) And (c<d) or (e<f)

if (a<b) goto

goto

if (c<d) goto

goto

if (e<f) goto

goto

if (a<b) and (c<d)

if a<b is true, control must continue second conduction

Backpatch statement 1 to statement 3.

if (a<b) goto 3.

goto 5

if c<d goto 7

goto 5

Translate (a<b) And (c<d) or (e<f) into three address Statements using back patching.

(a<b) And (c<d) or (e<f)

Step 1: Apply operator precedence AND has higher precedence than OR ~~so~~ expression

[(a<b) AND (c<d)] or (e<f)

Step 2: Generate incomplete jumps

1. if a<b goto -
2. goto -
3. if c<d goto -
4. goto -
5. if e<f goto -
6. goto -

The underscore represent a target address

Step 3. Handle AND

For (a<b) AND (c<d) if a<b is true for (a<b) AND (c<d), if control must continue to the 2nd condition.

Backpatch statement 1 to statement 3.

1. if a<b goto 3.

Step 4: Handle OR

If the First AND expression is false, the second or operand e<f must be evaluated.

Backpatch statement 2 to statement 5.

Backpatch statement 4 to statement 5.

Step 5:

Final backpatched code

1. if $a<b$ goto 3
2. goto 5.
3. if $c<d$ goto 7
4. goto 5
5. if $e<f$ goto 7
6. goto 8
7. TRUE
8. FALSE

Backpatching means generating pump instructions 1st and filling target address.

---

construct three address code using the following production for Boolean expression $(a>b)$ or $(c<d)$ or $(e<f)$

$$B \to B_1 \; || \; B_2$$
$$B \to B_1 \; \&\& \; B_2$$
$$B \to \; ! \; B_1$$
$$B \to E_1 \text{ relop } E_2$$
$$B \to \text{true}$$
$$B \to \text{false}$$

Step 1: $[(a>b) \text{ OR } (c<d)] \text{ OR } (e<f)$

Step 2:

L1:
if $(a>b)$ goto LTRUE
goto L2

Step 3:
check the 2nd condition
L2: if $c<d$ goto LTRUE
goto L3

Step 4: check the third condition
L3: if $e<f$ goto LTRUE
if $e<f$ goto LTRUE
got: LFalse

Final TAC for Q3
L1: if $a>b$ goto LTRUE
gotoL2

4. Generate the three address code for the following program fragment while $[A<c]$ and $[B>D]$ do if $A=1$ then $C=C+1$ else while $A<=D$ do
$A = A+B$

complete TAC
L1: if $A<C$ goto L2
goto LEND
L2: if $B>D$ goto L3

goto LEND

L3: if A==1 goto L4

goto L5

L4: $t_1 = c + 1$

$c = t_1$

goto L_1

L5: if A<=D goto L_6

goto L_1

L_6: $t_2 = A + B$

$A = t_2$

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	15%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps